WIRELESS & MOBILE SECURITY · ML-BASED WIDS / WIPS PROTOTYPE

CATCHING THE DEAUTH FLOOD

A Random Forest that reads Wi-Fi traffic one second at a time, raises an alert the moment a deauthentication flood begins, and logs the sender's MAC address.

DANICK KIRENGA · 192032 · BSC COMPUTER NETWORKS & CYBERSECURITY · STRATHMORE UNIVERSITY

HEADLINE RESULT · PLATE I

99.3%

accuracy on AWID3, a Wi-Fi network the model never saw. Trained only on the WPA3 capture and applied with no retraining.

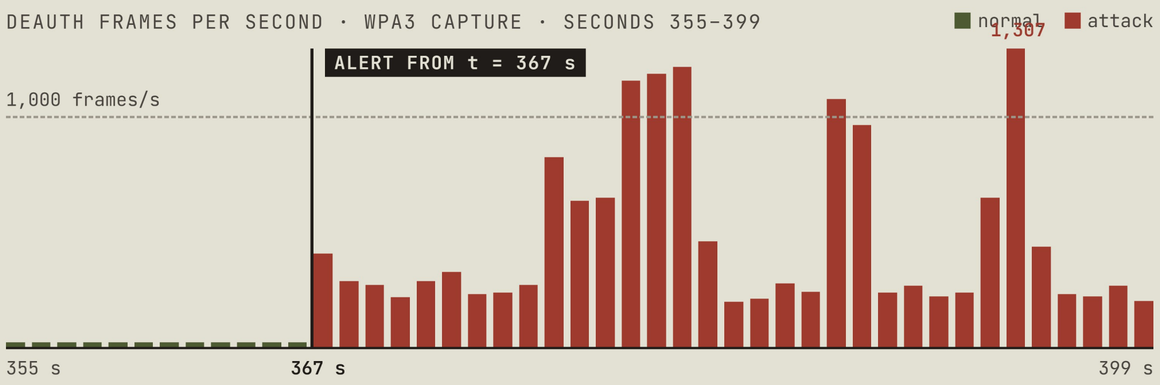
0 false alarms in 423 normal seconds

176 / 180 attack seconds caught

01 · THE ATTACK

"Disconnect!", shouted hundreds of times a second

A deauthentication frame (802.11 management frame, type 0, subtype 12) tells a device to drop its connection. An attacker forges them in bulk, posing as the access point or the client, so every device keeps getting kicked off.



02 · HOW IT WORKS

Six steps from raw frames to an alert

1 · READ
Wireless traffic
CSV rows, or a pcap read with Scapy

→

2 · SLICE
1-second windows
frames grouped by their timestamp

→

3 · MEASURE
Six features
computed per window, listed below

4 · CLASSIFY
Random Forest
100 decision trees vote: normal or attack

→

5 · DETECT
Raise an alert
an [ALERT] line for every attack second

→

6 · PREVENT (WIPS)
Log the MAC
top deauth sender written to blocklist.log

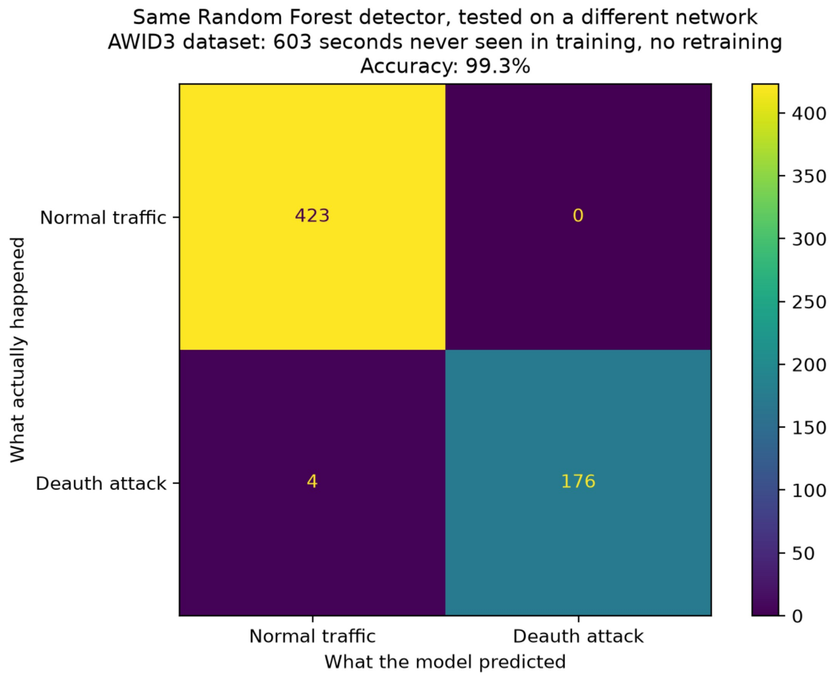
THE SIX FEATURES, MEASURED EVERY SECOND

Total frames	Deauth frames	Deauth ratio
Avg signal strength (dBm)	Avg frame length	Retry ratio

03 · RESULTS

Tested where it counts: on a network it had never seen

PLATE I · INDEPENDENT TEST — AWID3 · 603 SECONDS · NO RETRAINING



Each square counts 1-second windows. Diagonal = correct. Top right = false alarms. Bottom left = missed attacks.

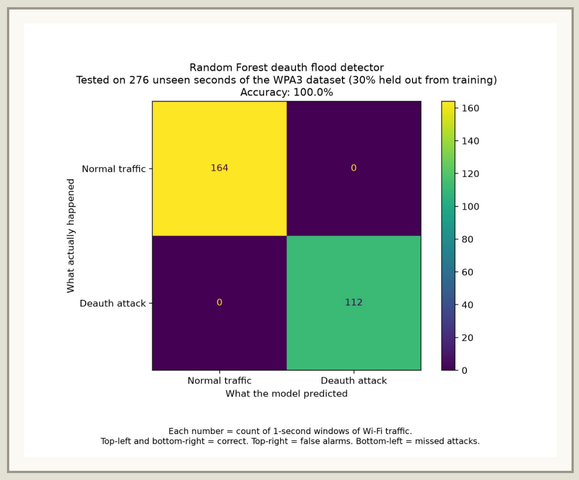


PLATE II · SAME DATASET

100%

Held-out WPA3 test

276 seconds the model never trained on (a random 30% held out). Expected: normal traffic in this capture holds zero deauth frames, so the deauth count alone separates the two classes.

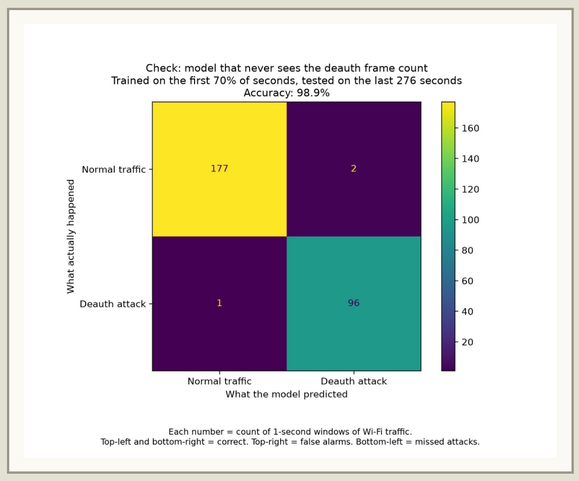


PLATE III · SANITY CHECK

98.9%

Without the deauth count

A separate model, trained on the first 70% of seconds and tested on the last 276. Frame volume, size, signal and retries alone still give the attack away.

04 · IN ACTION

The alert fires on the first attack second

```
> py detector.py Deauth.pcap --start 360 --seconds 20
t=364s normal (800 frames)
t=365s normal (513 frames)
t=366s normal (561 frames)
[ALERT] t=367s deauth flood: 407 deauth frames, top sender 02:00:00:00:00:cb
[WIPS]  logged 02:00:00:00:00:cb to blocklist.log
[ALERT] t=368s deauth flood: 286 deauth frames, top sender 02:00:00:00:00:72
[WIPS]  logged 02:00:00:00:00:72 to blocklist.log
[ALERT] t=369s deauth flood: 273 deauth frames, top sender 02:00:00:00:00:72
…an alert on every second through t=379s
```

Replay of the recorded WPA3 capture through detector.py. With no Wi-Fi adapter available, traffic is replayed from file rather than sniffed live.

05 · LIMITS & NEXT STEPS

What it can't do yet

LIMITS

- Few harmless deauths to learn from: normal traffic held 0 (WPA3) and 3 (AWID3), so false alarms on a busy real network are untested.
- Floods slower than any in training slip past: all 4 AWID3 misses had 45–78 frames/s; training never went below 106.
- Senders are spoofed: logged MACs are locally administered (02:…), so blocking by MAC could block the real access point.

WHAT I WOULD IMPROVE

- Train on both datasets together
- Capture live in monitor mode on a lab network
- Also detect disassociation and beacon floods
- Enable Protected Management Frames (802.11w), which WPA3 requires

06 · AI USE

How I used AI

I used Claude (Anthropic) as a coding assistant to plan the build, draft train.py, detector.py and eval_awid3.py, and debug errors. I ran every step myself, checked results against the dataset labels, and can explain how each part of the code works.

BUILT WITH

Python · scikit-learn · pandas · Scapy · matplotlib

TRAINING DATA

WPA3 Attacks Dataset (Mendeley Data), Deauthentication capture. 722,606 frames → 917 one-second windows: 546 normal, 371 attack.

INDEPENDENT TEST

AWID3, files Deauth_0 to Deauth_32. 1,626,469 frames → 603 windows: 423 normal, 180 attack. Model applied exactly as trained.

CODE

train.py builds features and trains · detector.py replays a pcap, alerts and logs MACs · eval_awid3.py tests on AWID3